# (API) The term is an acronym for “application programmable interface.”

This is in fact the interface that gives a person or a system the ability to interact with a piece of hardware or software through the operations that are exposed. The “user” of an API might be a system or a developer.

APIs are everywhere. A mobile app, a weather app, your laptop’s operating system, a fridge, your smart doorbell, and even your car all expose or use APIs.

We will focus more on APIs that run on the web over the HTTP protocol, serving data for web applications in JSON format

# Some types of API

1- Push/ Stream API: Event driven (web server will “push” messages or notifications to a web browser). It is used when time, real-time processing are important.
2- Native APIs: Interface of a device or a tool, allowing us to interact with the device or tool.
    a. Example web browser Javascript API that allow execution of javascript.
    b. Example accessing our smartphone’s camera through the exposed native API.
3- SDKs: Software Development Kit, is a package of tools written in different languages that helps developers build applications on top of that package.
    **a.** .NET SDK provides a set of libraries and tools that allows developers to build their own APIs or web applications. SDK sometimes comes as part of Integrated Development Environment (IDE) and give you scaffolding capabilities. **Scaffolding** is the code generation capability given by some tools or frameworks. It aids the development process by autogenerating code based on predefined options or by taking in some input.
4- REST : Representational state transfer, is an architectural style in the API world that comes with a set of principles and guidelines about how to design your API and leverage the underlying protocol in the correct way.
5- RPC APIs: Remote Procedure Call. Similar to calling methods from application and using classes but these methods are hosted on a different machine over the network.

## Web Apps and ASP.NET MVC framework:

Most of Web App have frontend and backend code intertwined or have structures or components that generate HTML code from the server. Such an example can be seen with

the ASP.NET MVC framework, where structures like Tag Helpers and HTML Helpers do exactly this: generate HTML from the server. Example (@Html.TextBoxFor(x=>x.AddedDate))

Separating backend from frontend benefits:

- You can scale only the part of your app that needs scaling.
- You can have different technologies, such as modern frameworks like React or Angular on the frontend, and use powerful languages for the backend.
- You can have separate, specialized, and dedicated teams.
- You can minimize merge conflicts.
- You can have several backends serving one UI.
- You can reuse the backends to serve different types of frontends (mobile apps, web, mobile).
- You can decrease costs and keep them under control.

## What is a WEB API?

The ASP.NET Web API is a set of libraries that allows us to develop RESTful HTTP APIs (backends) using .NET, C#, or the language we prefer to work with, in .NET.

"JSON and XML are data formats that allow APIs to send responses."

**Extra note (useful for exams / teaching)**

- **JSON** is lightweight and most commonly used today (especially with REST APIs).
- **XML** is older, more verbose, but still used in some enterprise and SOAP-based APIs.

Web API gets delivered with the Visual Studio installer, and is available under *Workload* ➤ *Web & Cloud,* as illustrated in Figure 1-6.

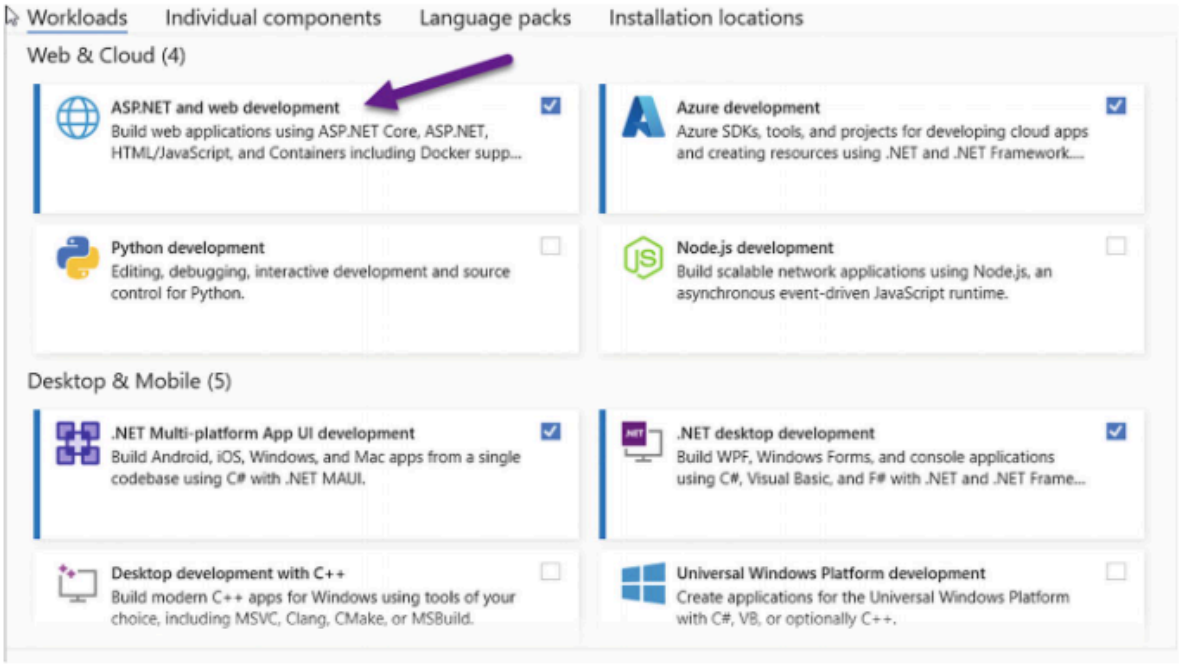


***Figure 1-6*** Visual Studio Workload—ASP.NET and Web development